

# Especificaciones del Proyecto Final: Sistema de Transferencias Bancarias

## 1. Introducción y Objetivo del Proyecto

El objetivo de este proyecto final es diseñar, desarrollar e implementar una aplicación de simulación bancaria dedicada a la ejecución de transferencias monetarias seguras. Los estudiantes aplicarán los pilares de la computación en la nube: alta disponibilidad, escalabilidad horizontal y automatización mediante una arquitectura basada en la infraestructura de Amazon Web Services (AWS).

## 2. Arquitectura de Infraestructura y Requisitos Técnicos

El diseño del sistema debe alinearse estrictamente con la topología y tecnologías solicitadas a continuación:

### A. Componentes y Estrategia de Contenedores

- **Capa del Backend:** Toda la lógica de negocio, enrutamiento de API, manejo de transacciones y seguridad se consolidará en una única aplicación monolítica contenedorizada.
- **Capa del Frontend:** La interfaz de usuario se desacopla en su propio contenedor independiente para facilitar su distribución.

### B. Orquestación y Despliegue en AWS (EC2 & Docker Swarm)

El despliegue se realizará sobre servidores virtuales de AWS utilizando un clúster de Docker Swarm distribuido de la siguiente manera:

- **Instancias para el Backend (Mínimo 2 instancias EC2):** El contenedor del backend monolítico debe replicarse y ejecutarse en al menos dos instancias EC2 distintas que actúen como nodos trabajadores/administradores en Swarm. Esto garantizará la alta disponibilidad y la tolerancia a fallos en caso de que un nodo quede fuera de servicio.
- **Instancia para el Frontend (Mínimo 1 instancia EC2):** La interfaz web se desplegará en una instancia EC2 dedicada, comunicándose con el clúster de backend a través de la red superpuesta (overlay network) de Swarm o mediante un punto de enlace configurado.

### C. Persistencia y Base de Datos (Servicios Administrados Externos)

Para la persistencia de datos, las instancias del backend deberán conectarse simultáneamente a una plataforma de datos centralizada externa. Los estudiantes deberán integrar de forma obligatoria al menos uno de los siguientes servicios administrados de bases de datos:

- **MongoDB Atlas:** Para enfoques orientados a documentos NoSQL, garantizando alta disponibilidad a través de conjuntos de réplicas completamente gestionados.
- **Aiven:** Plataforma administrada para desplegar bases de datos de código abierto (como

- PostgreSQL, MySQL o Redis) con respaldo automático y monitoreo integrado.
- **Supabase:** Plataforma Backend-as-a-Service basada en PostgreSQL administrado que provee capacidades relacionales inmediatas, autenticación y tiempo real.

## D. Automatización CI/CD

- Al realizar un *Push Request* o actualización en la rama principal, un pipeline automatizado construirá las imágenes de Docker actualizadas, las subirá al registro correspondiente y ordenará al clúster de Docker Swarm realizar un despliegue progresivo (rolling update) sin caída del servicio.

# 3. Descripción de Módulos y Funcionalidades de la Aplicación

La aplicación bancaria debe estructurarse bajo los siguientes módulos funcionales detallados:

## Módulo 1: Gestión de Usuarios y Autenticación

- **Registro de Clientes:** Formulario para la creación de nuevas cuentas bancarias solicitando datos personales básicos, correo electrónico y contraseña encriptada. En este punto se dispara la lógica de asignación automática del número de cuenta único de 10 dígitos.
- **Inicio de Sesión Seguro:** Autenticación basada en sesiones o tokens (por ejemplo, JWT) manejada de forma centralizada por el monolito.
- **Perfil de Usuario:** Consulta y actualización de datos generales y visualización del número de cuenta único asignado por el sistema.

## Módulo 2: Consultas Financieras y Posición Global

- **Tablero Principal (Dashboard):** Al ingresar, el usuario debe ver de forma clara su saldo disponible actual en moneda local.
- **Historial de Movimientos:** Lista cronológica de todas las transacciones realizadas (tanto abonos como cargos). Cada registro debe mostrar la fecha/hora, concepto, monto y la cuenta origen/destino.

## Módulo 3: Motor de Transferencias Bancarias (Módulo Crítico)

- **Registro de Cuentas Destino:** Funcionalidad para dar de alta cuentas de terceros usando el número de cuenta único validado previamente y un alias.
- **Ejecución de Transferencias:** Formulario para especificar la cuenta destino, el monto a transferir y un mensaje.
- **Lógica de Validación y Atomicidad:** El backend debe validar que la cuenta origen tenga fondos suficientes antes de proceder. La operación de restar saldo a la cuenta A y sumar saldo a la cuenta B debe ejecutarse dentro de una **transacción ACID** o un patrón de consistencia equivalente en el servicio administrado (MongoDB Atlas / Aiven / Supabase) para asegurar la integridad financiera en caso de fallos.

## Módulo 4: Notificaciones y Auditoría

- **Bitácora de Eventos:** Registro interno en el backend de todas las peticiones críticas con fines de auditoría. Se creará un log en base de datos con la siguiente información: fecha y hora, usuario involucrado, acción realizada (login exitoso, login fallido, transferencia aprobada, transferencia aceptada, alta de cuenta, etc), estado del evento (exitoso, fallido, pendiente), detalle(información relevante, si fue un inicio de sesión, la cuenta involucrada, para las transferencias considerar cuenta origin, destino, monto, etc).
- **Alertas de Movimientos:** Notificación visual o confirmación en la interfaz de usuario inmediatamente después de concluir una transferencia de manera exitosa o fallida.

## 4. Estructura y Reglas del Número de Cuenta

Para garantizar la integridad de las operaciones y simular un entorno bancario real, cada usuario registrado dispondrá de un identificador financiero único regulado por las siguientes especificaciones:

### A. Longitud y Formato Estático

El número de cuenta será de 10 dígitos numéricos, estructurado internamente en tres bloques definidos:

- **Prefijo de Origen (Dígitos 1-3):** Un código estático fijado en 180.
- **Identificador Secuencial (Dígitos 4-9):** Un bloque de 6 dígitos que corresponde de forma única y consecutiva al ID de registro del usuario en la base de datos (rellenado con ceros a la izquierda si es necesario).
- **Dígito Verificador (Dígito 10):** Un único dígito de control calculado a través de un algoritmo de validación interna.

### B. Algoritmo de número de cuenta (Paso a Paso)

Al completarse el registro de un nuevo cliente, el backend ejecutará de forma automática el siguiente flujo aritmético antes de almacenar los datos en la nube:

1. **Concatenación Base:** Se une el prefijo 180 junto con el ID secuencial formateado a 6 dígitos (ej. para el ID 34, la base intermedia es 180000034).
2. **Suma de Dígitos:** Se realiza la adición matemática individual de los 9 caracteres numéricos ( $1 + 8 + 0 + 0 + 0 + 0 + 0 + 3 + 4 = 16$ ).
3. **Cálculo Módulo 10:** Se obtiene el residuo de dividir dicha suma entre 10 ( $16 \bmod 10 = 6$ ). Si el residuo es exactamente 0, el dígito verificador se asigna como 0.
4. **Consolidación:** Se añade el dígito resultante al final de la cadena base para conformar el identificador definitivo (ej. 1800000346).

### C. Reglas de Negocio en el Backend

- **Inmutabilidad Absoluta:** Una vez que el sistema genera y asigna el número de cuenta durante el registro, este campo queda protegido; no puede ser editado ni transferido a otro usuario.
- **Restricción de Unicidad:** La base de datos administrada elegida debe contar con un índice de unicidad (unique: true) configurado explícitamente para este atributo,

previniendo duplicados ante peticiones simultáneas.

- **Optimización mediante Expresiones Regulares:** Antes de procesar cualquier consulta o transferencia en la base de datos, el backend debe validar mediante una expresión regular (`/^\d{10}$`) que la cuenta destino cumpla con la longitud y formato requeridos, rechazando peticiones malformadas de inmediato.

## 5. Rúbrica de Evaluación por Niveles de Desempeño

El proyecto se evaluará mediante los siguientes criterios y niveles de logro.

| Componente (Peso)                              | Excelente (100%)                                                                                                                                                                    | Bueno (80%)                                                                                                                                       | Suficiente (60%)                                                                                                                                                            | Insuficiente (0%)                                                                                                      |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Arquitectura Monolítica y Módulos (25%)</b> | Los 4 módulos funcionan correctamente junto con la generación automática y validación estricta del número de cuenta de 10 dígitos. El motor maneja transacciones atómicas íntegras. | Funcionan los módulos básicos y la generación de cuentas, pero se presentan ligeras fallas de validación o consistencia visual en transferencias. | Falta alguno de los módulos secundarios (auditoría o notificaciones) o el número de cuenta se genera sin seguir las reglas estipuladas. Lógica de transferencias inestable. | La aplicación no compila, no es un backend monolítico o carece de la lógica financiera y de control de cuentas mínima. |
| <b>Configuración de Docker Swarm (20%)</b>     | Clúster Swarm perfectamente inicializado; uso correcto de redes de tipo overlay y replicación nativa de servicios.                                                                  | El clúster está activo, pero la asignación de réplicas o la comunicación en la red overlay presenta intermitencias.                               | Los servicios corren en Docker, pero no aprovechan las características nativas de orquestación de Docker Swarm.                                                             | No utiliza Docker Swarm ni estructura el proyecto mediante contenedores orquestados.                                   |
| <b>Topología en AWS EC2 (20%)</b>              | Despliegue distribuido real en AWS con la infraestructura mínima exacta (2 EC2 para backend, 1 EC2 para frontend).                                                                  | Desplegado en AWS EC2, pero la separación de instancias no cumple con la redundancia de infraestructura solicitada.                               | El entorno está alojado en una sola instancia virtual grande sin distribución ni balanceo real de capas.                                                                    | No se realizó el despliegue en instancias de Amazon Web Services (AWS).                                                |

| Componente<br>(Peso)                   | Excelente<br>(100%)                                                                                                     | Bueno (80%)                                                                                                                  | Suficiente (60%)                                                                                                          | Insuficiente (0%)                                                                               |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Persistencia Administrada (15%)</b> | Integración y conexión perfecta hacia MongoDB Atlas, Aiven o Supabase. Cero datos guardados localmente en contenedores. | Conexión exitosa al servicio administrado, pero con credenciales expuestas en código o fallos ocasionales de conexión.       | Uso de base de datos externa pero sin usar los servicios administrados requeridos (ej. BD corriendo en un VPS plano).     | Persistencia local dentro de los mismos contenedores volátiles (pérdida de datos al reiniciar). |
| <b>Pipeline de CI/CD (20%)</b>         | Automatización completa e impecable: el Push Request compila, genera la imagen y actualiza el Swarm en AWS sin caídas.  | El pipeline funciona al hacer push, pero requiere configuraciones manuales finales en el servidor para reflejar los cambios. | Pipeline configurado a medias; ejecuta pruebas o construye imágenes pero no automatiza el despliegue final en producción. | Ausencia de herramientas de integración continua; despliegue enteramente manual.                |

## 6. Entregables Obligatorios

- **Enlace al Repositorio de Código:** Deberá incluir todo el código fuente, los archivos de configuración de Docker y la definición del workflow de CI/CD.
- **Documento de Arquitectura:** Un breve reporte en PDF que incluya el diagrama de la infraestructura desplegada en la nube.

## 7. Revisión

La fecha de presentación del proyecto queda sujeta al día de aplicación del examen ordinario.